ARL Spec: Radio Telescope Array

Brendan Griffiths

Contents

1 Service overview 2
1.1 General state and routines 2
1.2 Message conventions 3
2 Transmitted data 4
2.1 Telemetry 4
2.2 State (retained) 5
2.3 Events 5
3 Scan images 6
3.1 Sample to pixel 6
3.2 Pixel value 7
4 Controls 7
5 Alerts 8
6 Configuration 9
7 Scenario events 10
A Reference images 12
A.1 Centaurus A (`tgt-cen`) 12
A.2 Orion A (`tgt-ori`) 13
A.3 Vela (`tgt-vel`) 13
A.4 Large Magellanic Cloud (`tgt-lmc`) 14

1 Service overview

The Radio Telescope Array (RTA) is a field of 100 radio dishes grouped into sub-arrays. A sub-array scans a target in the sky, and the samples its dishes return form an image of the target. It is the highest-rate service at the ARL. Topic prefix: `arl/rta`.

Entity kind	Ids	Notes
`dish`	`dish-001` to `dish-100`	10 rows of 10, numbered by row from the north-west. Each dish has `row`, `col`, `lat` and `lon` in `$topology`.
`subarray`	`main` plus up to 3 others	A group of dishes that scans together. Names match `[a-z][a-z0-9-]{0,15}`, may not start with `dish-` and may not be `global`. Sub-arrays appear and disappear, so watch `$topology`.
`service`	`global`	The array as a whole: active dish count, wind stow, alert summary.

1.1 General state and routines

- **Dishes in service**: at start-up every dish is in `main`. About 80 to 90 dishes are in service; the rest are disabled for maintenance (`enabled = false`).
- **Normal reporting**: every dish that is not scanning reports its condition every 120 s on the `normal` stream.
- **Scanning**: while a sub-array scans, each of its dishes reports once a second on the `scan` stream instead. A scan divides the target's field of view into a 128 × 128 grid and samples every grid cell 3 times with different dishes, sharing the work between the sub-array's dishes; with 90 dishes a scan takes about 9 minutes. The pointing offsets and signal strength of the samples are what build the image. A dish that drops out of a scan leaves its remaining samples untaken.
- **Tracking**: during a scan the dishes follow the target across the sky, so their azimuth and elevation change continuously.
- **Targets**: a target is either an id from the ARL API target catalogue or a custom sky position `{ra_rad, dec_rad, fov_rad}`. A scan needs its target at least 15° above the horizon to start, and fails if the target sets below 5°.
- **Observation frequency**: each sub-array observes at a frequency (default 1.42 GHz). Targets look different in different bands.
- **Autonomous schedule** (live deployment): the RTA plans its own scans every 45 to 90 minutes on the `main` sub-array, announced 5 minutes ahead through `next_scan` and `scan_state = scheduled`. About one scan in four also splits off 20 to 40 dishes into a temporary sub-array `auto-2` to scan a second target at the same time; those dishes return to `main` afterwards. The schedule never overrides a sub-array a researcher has commanded in the last 2 hours. Scheduled scans have `"origin": "schedule"` in their scan events.
- **Wind stow**: when the gust at the array exceeds 15 m/s for 60 s, every dish stows (points straight up), running scans pause, and `wind_stow` becomes `true`. It releases once the gust has stayed below 10 m/s for 10 minutes. Paused scans then wait to be resumed.
- **Dish mechanics**: azimuth 0 to 2π (from north, wrapping), elevation 0.0873 to 1.5708 rad (5° to 90°). Azimuth slews at 2°/s, elevation at 1°/s.
- **Load**: under 1 message/s when idle; about 90 messages/s while `main` scans with 90 dishes.

1.2 Message conventions

Every service follows the same MQTT layout. `<entity>` is an entity id (or `global` for the whole service).

Topic	Retained	Content
`arl/rta/$topology`	yes	Machine-readable list of the entities, streams (with field names, types, units and periods), state items, events and controls of the service. Re-published when it changes.
`arl/rta/<entity>/telemetry/<stream>`	no	Periodic measurements.
`arl/rta/<entity>/state/<item>`	yes	The current value of a state item, re-published when it changes.
`arl/rta/events/<kind>`	no	One-off events, including `alert`.
`arl/rta/cmd/<entity>/<control>`	no	Commands sent by the platform.
`arl/rta/ack/<command_id>`	no	Acknowledgements of a command.

Telemetry and event payloads carry an envelope: `v` (schema version, integer, currently 1), `ts` (simulated time, Unix milliseconds), `seq` (per-topic sequence number, restarts with the service; gaps reveal lost messages) and `entity` (entity id). The measurement fields are added next to the envelope. A measurement that cannot be taken is `null`. State payloads are `{"v", "ts", "entity", "item", "value"}`.

Commands are JSON `{"command_id": <string>, "researcher": <researcher code>, "value": <value>}`. The researcher must be verified by the ARL API for the service. Acknowledgements carry `command_id`, `target`, `control`, `researcher`, `state` (`accepted`, `rejected`, `in_progress`, `completed`, `failed`), `progress` (0 to 1 for long-running controls), `reason` and `detail`. Rejection and failure reasons: `controls_disabled`, `malformed`, `unknown_control`, `unknown_target`, `unknown_researcher`, `researcher_defunct`, `researcher_not_permitted`, `api_unavailable`, `invalid_value`, `precondition_failed`, `busy` (the target is already running a long command), `fault`, `timeout`, `interrupted`.

Alerts are events on `arl/rta/events/alert` with `alert_id`, `type` (the alert key below), `severity` (`info`, `warning`, `critical`), `state` (`raised` or `cleared`), `value`, `limit`, `unit`, `message` and `metadata`. An alert that escalates is raised again with the higher severity under the same `alert_id`. Limits come from the ARL API (`/v1/alert-limits/` + `rta`).

Every service also publishes these:

Measurement	Key	Level	Type / unit	Description
Online	`online`	Global	boolean	Retained state. `true` while the service runs, `false` when it stops or loses its broker connection (last will).
Active alerts	`active_alerts`	Global	array of alert ids	Retained state. The ids of the alerts currently raised, in the order they were raised.

2 Transmitted data

2.1 Telemetry

2.1.1 Normal (`arl/rta/<dish>/telemetry/normal`, every 120 s, dishes not scanning)

Measurement	Key	Level	Type / unit	Description
Temperature	`temperature_k`	Dish	float, kelvin	Temperature of the dish structure. Rises in sunshine and while slewing.
Elevation	`elevation_rad`	Dish	float, radians	Current elevation angle above the horizon.
Azimuth	`azimuth_rad`	Dish	float, radians	Current azimuth angle, clockwise from north.
Power draw	`power_w`	Dish	integer, watts	Electrical power the dish is using. Higher while slewing and scanning, lower when stowed or disabled.
System temperature	`system_temperature_k`	Dish	float, kelvin	Receiver noise temperature. Lower is better; it grows as the dish's calibration ages and at low elevation.
Fault code	`fault_code`	Dish	enum	`none`, `drive_stall`, `overheat`, `receiver_fault`, `power_fault`, `comms_loss` or `encoder_error`.
Status	`status`	Dish	boolean	`true` when the dish is enabled and has no fault (or only `encoder_error`), i.e. in service.
Heartbeat	`ts` (envelope)	Dish	timestamp, ms	Time of the dish's latest message.

2.1.2 Scan (`arl/rta/<dish>/telemetry/scan`, every 1 s, dishes in a scan)

All the fields of the normal stream, plus:

Measurement	Key	Level	Type / unit	Description
Scan id	`scan_id`	Dish	string	The scan the dish is working on, e.g. `scan-0007`.
Pointing offset (RA)	`pointing_offset_ra_rad`	Dish	float, radians	Right ascension offset of this sample from the target centre. `null` when the dish took no sample this second (e.g. still slewing onto the target, or paused).
Pointing offset (Dec)	`pointing_offset_dec_rad`	Dish	float, radians	Declination offset of this sample from the target centre. `null` when no sample.
Signal strength	`signal_strength`	Dish	float, relative units	Brightness measured at that offset. Together with the offsets it forms the image. `null` when no sample.
Signal-to-noise ratio	`snr_db`	Dish	float, dB	Quality of this sample. `null` when no sample.
Band	`band_hz`	Dish	float, Hz	The frequency the scan observes at.
Scan progress	`scan_progress`	Dish	float, ratio 0 to 1	Fraction of this dish's share of the scan that is done. Holds still while the scan is paused.

2.2 State (retained)

Measurement	Key	Level	Type / unit	Description
Commanded position	`commanded`	Dish	object `{azimuth_rad, elevation_rad}`, radians	Where the dish has been told to point. Differs from the reported position while slewing. Updated every 10 s while tracking a target.
Sub-array	`subarray`	Dish	string	The sub-array the dish belongs to.
Last calibration	`last_calibration`	Dish	timestamp, ms	Time of the dish's last successful calibration.
Enabled	`enabled`	Dish	boolean	Whether the dish is enabled.
Stowed	`stowed`	Dish	boolean	Whether the dish is stowed (by command or by wind stow).
Members	`members`	Sub-array	array of dish ids	The dishes in the sub-array.
Scan state	`scan_state`	Sub-array	enum	`idle`, `scheduled`, `running`, `paused`, `cancelled` or `complete`.
Target	`target`	Sub-array	string or object `{ra_rad, dec_rad, fov_rad}`	The sub-array's target: a catalogue id or a custom position (radians). `null` if none is set.
Observation frequency	`observation_frequency`	Sub-array	float, Hz	Frequency the sub-array observes at.
Next scan	`next_scan`	Sub-array	timestamp, ms	When the next scan is scheduled to start. `0` if none.
Scan id	`scan_id`	Sub-array	string	The current or most recent scan of the sub-array. `null` if none yet.
Active dishes	`active_dishes`	Global	integer, count	Number of dishes in service (`status = true`).
Wind stow	`wind_stow`	Global	boolean	Whether wind stow is engaged.

When a sub-array is removed, its retained state topics are cleared (empty retained message).

2.3 Events

Measurement	Key	Level	Type / unit	Description
Scan event	`arl/rta/events/scan`: `scan_id`, `event`, `reason`	Sub-array	string, enum, enum	Scan lifecycle. `event` is `started`, `paused`, `resumed`, `completed`, `cancelled` or `failed`. `reason` is given for a pause caused by wind (`wind`) and for a failure (`target_set`, `too_few_dishes`, `wind`, `no_target`, `unknown_target`).
Scan start details	`target`, `ra_rad`, `dec_rad`, `fov_rad`, `band_hz`, `grid_size`, `overlap`, `dishes`, `expected_duration_s`, `origin`	Sub-array	target, radians, radians, radians, Hz, integer, integer, array	Sent with `started`: the target and its position and field of view, the band, the image grid size (cells per side), how many times each cell is sampled, the dishes taking part, the expected duration, and whether a researcher (`command`) or the schedule (`schedule`) started it.

Measurement	Key	Level	Type / unit	Description
			of dish ids, integer seconds, enum	
Alert	`arl/rta/events/alert`	Global / Dish	alert object	Alert raised or cleared (see *Alerts*).

3 Scan images

The radio telescope produces a 2D image from the scan data. A scan divides the target's field of view into a square grid of $N \times N$ pixels, and every sample on the `scan` stream measures the brightness of one pixel.

The grid comes from the scan's `started` event (`arl/rta/events/scan`) and the samples from the dishes' `scan` streams (matched by `scan_id`):

Symbol	Source	Meaning
N	`grid_size`	Pixels per side of the image (default 128).
θ	`fov_rad`	Field of view: the width and height of the imaged patch of sky, in radians.
k	`overlap`	How many samples (from different dishes) each pixel receives (default 3).
$\Delta\alpha, \Delta\delta$	`pointing_offset_ra_rad`, `pointing_offset_dec_rad` (scan stream)	Where in the field the sample was taken, relative to the target centre, in radians.
s	`signal_strength` (scan stream)	The brightness measured by the sample.

3.1 Sample to pixel

Each sample is taken at the centre of one pixel. The pixel (x, y) of a sample, with x the column (0 at the left) and y the row (0 at the top), is

$$(x, y) = \left(\left\lfloor N\left(\frac{\Delta\alpha}{\theta} + \frac{1}{2}\right) \right\rfloor, \left\lfloor N\left(\frac{1}{2} - \frac{\Delta\delta}{\theta}\right) \right\rfloor \right)$$

so x grows with the right ascension offset and y grows downwards as the declination offset decreases (north is at the top of the image). Equivalently, the offsets of the centre of pixel (x, y) are

$$(\Delta\alpha, \Delta\delta) = \left(\left(\frac{x + \frac{1}{2}}{N} - \frac{1}{2} \right)\theta, \left(\frac{1}{2} - \frac{y + \frac{1}{2}}{N} \right)\theta \right)$$

Offsets range over $\left[-\frac{\theta}{2}, \frac{\theta}{2}\right]$; for an even N (such as 128) the target centre falls between the four middle pixels.

3.2 Pixel value

A pixel's brightness is built from the samples that land on it:

$$I(x, y) = \frac{1}{n} \sum_{i=1}^{n} s_i, \quad n \le k$$

the mean `signal_strength` of the n samples of that pixel. Samples whose offsets or signal are `null` (the dish was still acquiring the target, or the scan was paused) carry no pixel and are skipped. A pixel with no samples (for example because its dish dropped out of the scan) is a hole in the image.

To display the image, scale I to a colour range, e.g. linearly from the minimum to the maximum pixel value of the scan. Brighter pixels are stronger radio sources. Different bands (`band_hz`) of the same target show different structure.

> **Note: the scan data is noisy.** Real telescope data is never clean, and neither is the ARL's. Every sample contains random noise, dishes differ slightly in sensitivity, and faulty dishes can produce outliers far brighter than the rest of the image, or no data at all (see `fault_code` and `snr_db`). An image built directly from the raw samples may therefore look grainy, patchy or almost completely dark.
>
> Do not assume your first image is correct. Expect to process and clean up the samples before (or while) building the image, and check your result, for example against published images of the catalogue targets.

4 Controls

Control	Key	Level	Value (type / unit)	Description
Start Scan	`start_scan`	Sub-array	boolean, must be `true`	Starts a scan now, or resumes a paused scan. Needs a target at least 15° above the horizon, at least 3 dishes in service and no wind stow. `completed` once the scan has started; the ack `detail` is the scan id.
Schedule Scan	`schedule_scan`	Sub-array	timestamp, ms (`0` clears)	Sets Next Scan; at that time it behaves as Start Scan (a scan that cannot start then ends `failed`). Must be in the future. Not while scanning.
Pause Scan	`pause_scan`	Sub-array	boolean, must be `true`	Pauses a running scan. The dishes keep reporting on the scan stream with unchanged progress.
Cancel Scan	`cancel_scan`	Sub-array	boolean, must be `true`	Cancels a running, paused or scheduled scan.
Set Target	`set_target`	Sub-array	catalogue id (string) or object `{ra_rad` 0 to 2π, `dec_rad`	Sets what the sub-array observes. Unknown ids are rejected. Not while scanning.

Control	Key	Level	Value (type / unit)	Description
			$-\pi/2$ to $\pi/2$, `fov_rad` 0.0005 to 0.05}, radians	
Set Observation Frequency	`set_observation_frequency`	Sub-array	float, Hz, 0.5e9 to 50e9	Sets the band the sub-array observes. Not while scanning.
Assign Sub-array	`assign_subarray`	Dish	string, sub-array name	Moves the dish to a sub-array. A new name creates the sub-array (at most 4 in total, otherwise `limit_reached`). A sub-array other than `main` that becomes empty is removed. Not while the dish is scanning.
Adjust Azimuth	`adjust_azimuth`	Dish	float, radians, 0 to 2π	Slews the dish to a new azimuth (`in_progress` until it arrives). Not while scanning or stowed. Fails on a stalled drive, or times out if the slew takes 1.5 times longer than expected.
Adjust Elevation	`adjust_elevation`	Dish	float, radians, 0.0873 to 1.5708	Slews the dish to a new elevation. As for Adjust Azimuth.
Enable Dish	`enable_dish`	Dish	boolean	Enables or disables the dish. Disabling a scanning dish removes it from the scan.
Stow Dish	`stow_dish`	Dish	boolean	`true` points the dish straight up and holds it there; `false` releases it (rejected during wind stow). A dish stowed by command leaves any scan.
Request Recalibration	`request_recalibration`	Dish	boolean, must be `true`	Calibrates the dish (120 s `in_progress`), resetting its system temperature drift. Not while scanning. May end `failed` with detail `calibration_failed`.

5 Alerts

Dish alerts carry `metadata` with the dish's `subarray`, `fault_code`, `row`, `col` and `scan_id` (when scanning).

Alert	Key	Level	Severity, limit key (default, unit)	Description
Availability	`availability`	Global	critical; `rta.availability.min_active` (80 dishes)	Fewer dishes in service than the minimum for 60 s. Clears at or above the minimum for 60 s.
Wind Stow	`wind_stow`	Global	warning; `rta.wind_stow.gust_ms` (15 m/s), `rta.wind_stow.release_ms` (10 m/s)	Wind stow is engaged (gust above the limit for 60 s). Clears when it releases (gust below the release limit for 10 min).
Dish Offline	`dish_offline`	Dish	warning; no limit	The dish has stopped communicating or lost power. Clears when it recovers.

Alert	Key	Level	Severity, limit key (default, unit)	Description
Dish Temperature	`dish_temperature`	Dish	warning, critical; `rta.dish_temperature.warn_k` (333 K), `rta.dish_temperature.crit_k` (343 K)	Dish temperature above the warning or critical limit for 120 s. Clears 5 K below the warning limit.
Dish Power	`dish_power`	Dish	warning; `rta.dish_power.max_w` (5000 W)	Power draw above the limit for 120 s. Clears below it.
Dish Movement	`dish_movement`	Dish	warning; no limit	A slew timed out or the drive stalled. Clears on the next successful slew.
Dish Calibration	`dish_calibration`	Dish	info, warning; `rta.calibration.max_age_h` (168 h)	Info when the last calibration is older than the limit (`metadata.last_calibration`); warning when a calibration failed. Clears on a successful calibration.
Dish Scan	`dish_scan`	Dish	warning; `rta.scan.min_snr_db` (3 dB)	During a scan: the dish left the scan, has a fault, sent no samples for 30 s, or its mean SNR over the last 60 s is below the limit. Also raised for a minute after a scan fails because its target set. Clears when the scan ends or the dish recovers.

6 Configuration

The service reads a TOML file (`config/` + `rta.toml` in the bundle, `--config` or `ARL_CONFIG` for a binary). Environment variables override the file, and command-line options override both (see *Running the ARL*). Defaults are the values in the provided file.

6.0.1 Common keys

Key	Type / unit	Default	Description
`seed`	integer	`1`	Random seed. The same seed, start time, scenario and commands give the same data. Env `ARL_SEED`.
`speedup_multiplier`	float, ×	`1.0`	Simulation speed relative to real time, up to the service's `max_speedup`. Env `ARL_SPEEDUP`.
`control_enabled`	boolean	`true`	Whether the service accepts commands. When `false` every command is rejected (`controls_disabled`). Env `ARL_CONTROL_ENABLED`.
`start_time`	`"now"` or RFC 3339	`"now"`	Simulated time at start-up. Env `ARL_START_TIME`.
`scenario`	string, path	`""`	Scenario file to load, empty for none. Env `ARL_SCENARIO` (compose file).
`mqtt.address`	string	`"mosquitto"`	MQTT broker host. Env `MQTT_BROKER_ADDRESS`.
`mqtt.port`	integer	`1883`	MQTT broker port. Env `MQTT_BROKER_PORT`.
`mqtt.user`	string	`""`	Broker username, empty for anonymous. Env `MQTT_BROKER_USER`.
`mqtt.password`	string	`""`	Broker password. Env `MQTT_BROKER_PASSWORD`.

Key	Type / unit	Default	Description
`mqtt.queue_capacity`	integer, messages	`200000`	Outgoing messages the service buffers while the broker is slow or unreachable.
`api.address`	string	`"api"`	ARL API host. Env `ARL_API_ADDRESS`.
`api.port`	integer	`8080`	ARL API port. Env `ARL_API_PORT`.
`api.require_researcher`	boolean	`true`	Whether commands must carry a researcher code the API accepts for this service.
`api.researcher_cache_ttl_s`	integer, s	`60`	How long a researcher check is remembered.
`api.alert_limits_refresh_s`	integer, s	`300`	How often the alert limits are re-read from the API.
`health.port`	integer	`9100`	Port of the service's `/healthz` endpoint (used by `healthcheck`). Env `HEALTH_PORT`.

6.0.2 RTA keys (`[rta]`)

Key	Type / unit	Default	Description
`num_dishes`	integer, 1 to 999	`100`	Number of dishes (`dish-001` upwards).
`active_dishes`	integer	`80`	Dishes guaranteed in service at start-up; a few more may be. The rest start disabled.
`scan_frequency_hz`	float, Hz	`1`	Reporting rate of the `scan` stream (whole seconds).
`normal_frequency_hz`	float, Hz	`0.00833`	Reporting rate of the `normal` stream (0.00833 Hz is every 120 s).
`max_subarrays`	integer	`4`	Most sub-arrays that may exist at once, including `main`.
`scan_grid_size`	integer, 1 to 1024	`128`	Cells per side of a scan's grid. Scan time grows with its square.
`scan_overlap`	integer, 1 to 10	`3`	How many times each grid cell is sampled.
`redistribute_on_fault`	boolean	`false`	Whether the samples of a dish that drops out of a scan are shared among the remaining dishes instead of being lost.
`min_target_elevation_deg`	float, degrees	`15`	Lowest target elevation at which a scan may start.
`geojson`	string, path	`"/assets/arl.geojson"`	Site map with the dish positions. Empty or missing leaves positions out of `$topology`.
`assets_dir`	string, path	`"/assets/rta"`	Folder of sky images for the catalogue targets.
`max_speedup`	float, ×	`50`	Highest `speedup_multiplier` the service accepts.

7 Scenario events

Events with `service = "rta"` (format in *Running the ARL*). Weather (`service = "env"`) also affects the RTA: a `storm` gust front engages wind stow.

Fault	Entity	Params (default)	Effect
`overheat`	Dish	`peak_k` (345 K), `recover_after`	The dish heats up towards `peak_k` (Dish Temperature, fault code `overheat`).

Fault	Entity	Params (default)	Effect
`drive_stall`	Dish	`recover_after`	The drive stops and slews fail (Dish Movement, fault code `drive_stall`).
`receiver_fault`	Dish	`recover_after`	The receiver misbehaves and scan data is poor (fault code `receiver_fault`, may raise Dish Scan).
`power_fault`	Dish	`peak_w` (7000 W), `recover_after`	Power draw rises towards `peak_w`, then the dish goes offline (Dish Power, Dish Offline).
`comms_loss`	Dish	`recover_after`	The dish stops reporting (Dish Offline).
`encoder_error`	Dish	`recover_after`	Fault code `encoder_error`; the dish stays in service. Cleared by recalibration.
`recalibrate`	Dish, or none for all	none	A technician calibrates the dish (or every dish without an `entity`).
`repair`	Dish	none	Ends the dish's fault.

A fault lasts for `duration`, or `recover_after` (`HH:MM:SS` or seconds) if given instead; with neither it lasts until `repair` or the matching control.

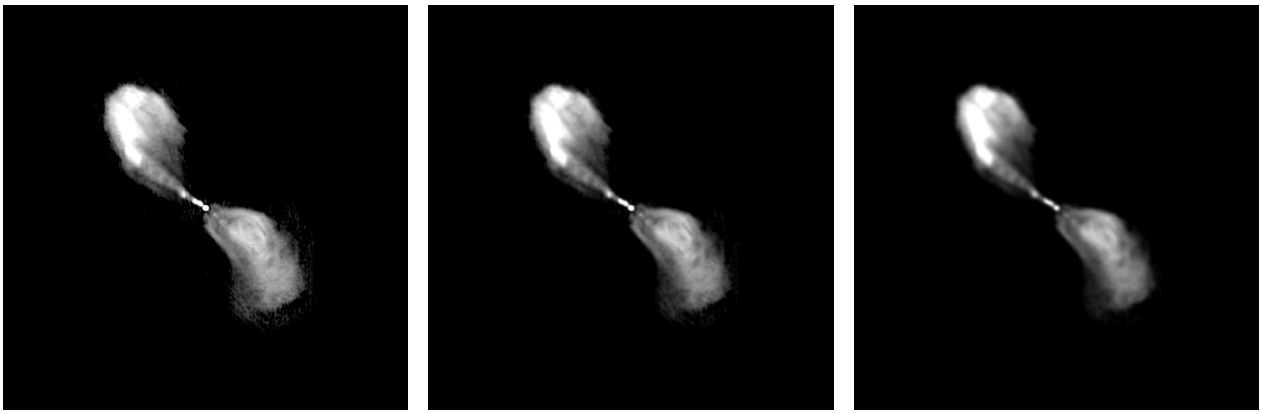
A Reference images

These are the correct images of the catalogue targets that have one, as an RTA scan of the target (at its catalogue position and `fov_rad`) should reproduce them, without noise or faults. Use them to check your reconstruction.

- Each image covers the target's field of view, oriented as in *Scan images*: pixel row 0 (top) is north (positive declination offset), and column 0 (left) is the most negative right ascension offset.
- Each target has three bands. A scan uses the band whose frequency range contains the sub-array's observation frequency, or else the band with the nearest centre: L (1 to 2 GHz, includes the default 1.42 GHz), C (4 to 8 GHz) and K (18 to 26 GHz).
- Brightness is shown from black (0) to white (1). Your image will have a different absolute scale (and an offset from the sky background), so compare shapes and relative brightness, not values.
- Fornax A (`tgt-for`) and PKS 1934-638 (`tgt-pks`) have no reference image.

A.1 Centaurus A (`tgt-cen`)

Field of view 0.010 rad.

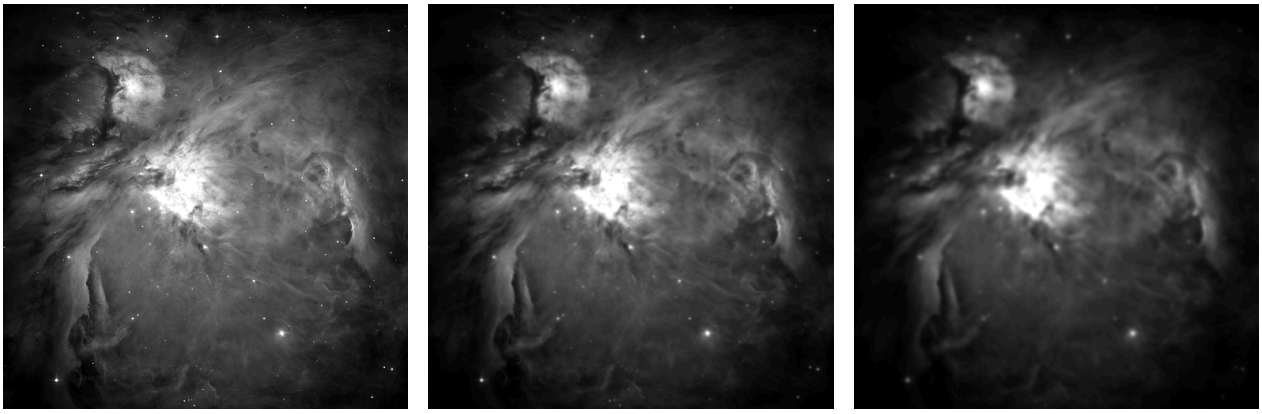


L band (1–2 GHz) C band (4–8 GHz) K band (18–26 GHz)

Credit: NRAO/AUI/NSF/Univ. Hertfordshire/M. Hardcastle, "Centaurus A in Radio" (VLA, 4.9 GHz), CC BY 4.0. Modified for COMS3011A: cropped, resized, split into bands, contrast-stretched, normalised, noise added.

A.2 Orion A (`tgt-ori`)

Field of view 0.010 rad.

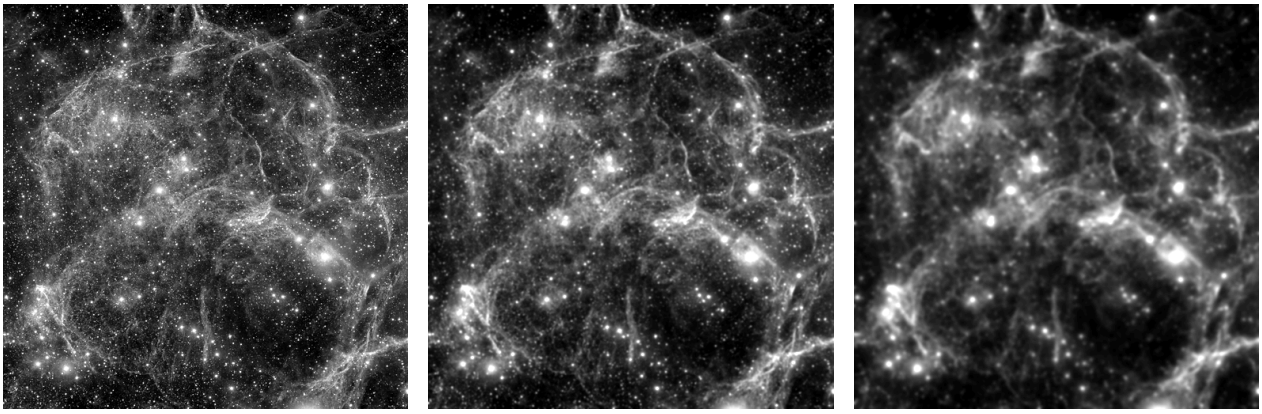


L band (1–2 GHz) C band (4–8 GHz) K band (18–26 GHz)

Credit: NASA, ESA, M. Robberto (STScI/ESA) and the Hubble Space Telescope Orion Treasury Project Team, ESA/Hubble heic0601a, CC BY 4.0. Modified for COMS3011A: cropped, resized, split into bands, contrast-stretched, normalised, noise added.

A.3 Vela (`tgt-vel`)

Field of view 0.020 rad.

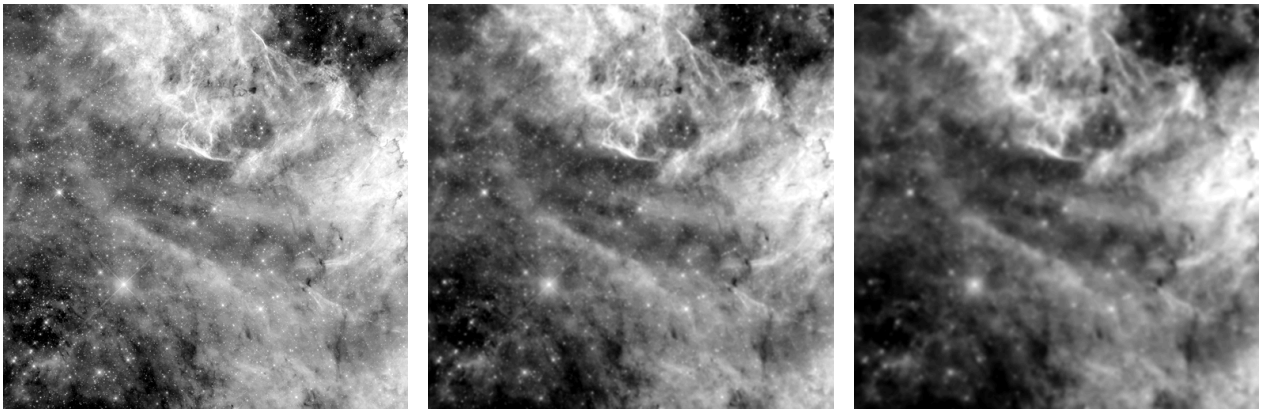


L band (1–2 GHz) C band (4–8 GHz) K band (18–26 GHz)

Credit: ESO/VPHAS+ team. Acknowledgement: Cambridge Astronomical Survey Unit, ESO eso2214a, CC BY 4.0. Modified for COMS3011A: cropped, resized, split into bands, contrast-stretched, normalised, noise added.

A.4 Large Magellanic Cloud (`tgt-lmc`)

Field of view 0.050 rad.



L band (1–2 GHz) C band (4–8 GHz) K band (18–26 GHz)

Credit: ESA/Hubble & NASA, C. Murray, "Capturing candyfloss clouds", ESA/Hubble potw2519a, CC BY 4.0. Modified for COMS3011A: cropped, resized, split into bands, contrast-stretched, normalised, noise added.

AI Declaration: Claude Code (Opus 5.5) - generated from the arl-sim source, reviewed